

closure_evidence_20260923

BOB-137 — closure evidence (partial): merge moved off the event-loop thread

Field	Value
Revision	1
Created	2026-09-23T15:22:23Z
Last modified	2026-09-23T15:22:23Z
Base commit	ffe3ed3 (changes uncommitted at time of writing)
Item status	NOT closed — live soak still owed (see “Honest gaps”)

1. What was still wrong (investigation before any change, §11.4.102)

The tracker row already records the root cause (1dd7b0a, 16 real stack dumps): `Deduplicator.merge_results()` runs as a plain synchronous call on the `asyncio` loop thread. BOB-145 (0572b71) made the merge ~17x cheaper but, by its own scope note, left the **call site** synchronous; the 2026-08-21 soak still measured 2/141 dead 7187 probes with the loop thread at `R/wchan 0`.

Re-read at HEAD ffe3ed3, there were **two** synchronous call sites on the loop thread, not one:

- `download-proxy/src/merge_service/search.py` (`_run_search`, formerly line 1080): `merged = self.deduplicator.merge_results(all_results)`

- `download-proxy/src/api/streaming.py _build_merged_update()` → `dedup.merge_results(raw)`, called from the SSE generator for every interim and final `merged_update` — once per open SSE client per 1.5 s throttle window, so it scales with client count.

A third defect only becomes live once the merge leaves the loop: `Deduplicator.merge_results` accumulated into `self._merged_groups`, rebinding it on entry. The orchestrator shares ONE `Deduplicator` across all searches, so two merges on worker threads would interleave and the first would append into — and return — the second's list.

2. Mechanism measured directly (real corpus from the BOB-145 guard, this host)

Command: `nice -n 19 ionice -c 3 .venv/bin/python <scratch>/gap_probe.py` (same `build_corpus` as `test_dedup_event_loop_blocking.py`, 5 ms heartbeat).

```
on-loop (pre-fix call shape)      N=400 merge_wall= 132.2ms
worst_gap= 135.4ms ticks_during_merge=1
to_thread (post-fix call shape)  N=400 merge_wall= 125.7ms
worst_gap= 10.8ms ticks_during_merge=13
on-loop (pre-fix call shape)      N=800 merge_wall= 283.5ms
worst_gap= 286.7ms ticks_during_merge=1
to_thread (post-fix call shape)  N=800 merge_wall= 311.8ms
worst_gap= 16.4ms ticks_during_merge=30
```

On the loop, the loop is frozen for the whole merge (worst gap == merge wall, one tick). Off the loop, the GIL switch interval hands the loop a turn every few ms; merge wall-clock is unchanged (this is latency isolation, not a speed-up).

3. RED — new test file against UNFIXED code

Test: `tests/unit/merge_service/test_bob137_merge_off_event_loop.py`

Command: `nice -n 19 ionice -c 3 .venv/bin/python -m pytest tests/unit/merge_service/test_bob137_merge_off_event_loop.py --import-mode=importlib -q -p no:cacheprovider`

E `AssertionError: the SSE generator re-merged ON the event-loop thread – every open SSE client freezes port 7187 for the merge duration (BOB-137).`

E `AssertionError: SearchOrchestrator._run_search ran Deduplicator.merge_results ON the event-loop thread – while it runs, port 7187 services no request (BOB-137).`

```
E      AssertionError: merge A returned groups from merge B
(['magnet:?xt=urn:btih:0000000000000000000000000000000000000064',
'magnet:?xt=urn:btih:0000000000000000000000000000000000000065',
'magnet:?
xt=urn:btih:0000000000000000000000000000000000000066']...) -
Deduplicator shares its working list across concurrent calls
FAILED tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_sse_merged_update_remerge
FAILED tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_run_search_merges_off_the
FAILED tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_interleaved_merges_on_one
3 failed, 3 passed in 4.41s
```

Each of the three defects fails its own test, for its own stated reason (the three passing cases are the sequential-behaviour characterization, which must pass both before and after). Note: a first RED run failed the characterization test too, but for a FIXTURE reason (near-identical titles fuzzy-merged); the fixture was corrected to dissimilar titles before the RED above was recorded.

Oracles: (1) structural — thread ident of the executing merge vs. the loop thread (load-independent); (2) user-observable — 5 ms heartbeat worst gap < 200 ms while a 400 ms GIL-holding merge runs; (3) deterministic forced interleave for re-entrancy.

4. Fix (minimal)

- `merge_service/search.py`: `merged = await asyncio.to_thread(self.deduplicator.merge_results, all_results)`.
- `api/streaming.py`: `_build_merged_update` split into `_snapshot_merge_inputs` (reads orchestrator state — runs ON the loop so the worker never iterates a `_tracker_results` dict the fan-out may be resizing) and `_merge_and_serialize` (pure; no orchestrator state). New `_build_merged_update_off_loop` does the snapshot on the loop and `await asyncio.to_thread(_merge_and_serialize, ...)`; both SSE call sites use it. The `sync_build_merged_update` keeps its signature and behaviour (existing stress tests call it directly).
- `merge_service/deduplicator.py`: `merge_results` accumulates into a local list, assigns `self._merged_groups` once at the end, returns the local list.

5. GREEN

```
tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_sse_merged_update_remerge
PASSED [ 16%]
tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_merge_output_unchanged_by
PASSED [ 33%]
tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_merge_output_unchanged_by
PASSED [ 50%]
tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_merge_output_unchanged_by
PASSED [ 66%]
tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_interleaved_merges_on_one
PASSED [ 83%]
tests/unit/merge_service/
test_bob137_merge_off_event_loop.py::test_run_search_merges_off_the
PASSED [100%]
```

```
===== 6 passed in 3.33s
=====
```

Regression: every test file under tests/unit + tests/integration that references deduplicator|merge_results|_run_search(|search_results_stream (includes the BOB-145 golden/order-independence guard, merged_update streaming, SSE disconnect, all-trackers-errored SSE contract), plus tests/stress/test_search_orchestration_stress_chaos.py:

```
252 passed in 32.91s          # targeted 13-file set incl. stress/
chaos
543 passed, 1 warning in 139.42s (0:02:19)  # full grep-selected
set, fixed tree
537 passed, 1 warning in 95.25s (0:01:35)    # same set on a
pristine HEAD worktree (baseline; 543-537 = the 6 new tests)
```

ruff check on the four changed/new files: All checks passed!

6. Honest gaps (§11.4.6)

1. **Live soak NOT run.** The item's acceptance criterion is a sustained-traffic soak against the running container with real tracker traffic (before/after curl timings on 7186/7187 + thread-state census). That needs live network and a ./start.sh --reload-python of the operator's running stack; it was not done here. Until it is, the row

must stay open. Expectation, UNCONFIRMED: the 2/141 dead-probe residual drops toward 0, but the >1 s stall tail may remain partially, because a GIL-holding worker still slows the loop (it no longer freezes it) and other synchronous work on the loop was not audited here.

2. **Other loop-thread CPU work not audited.** Only the two `merge_results` call sites were moved. `_serialize_merged_rows` in `GET /search/{id}` and any other synchronous per-request work on the loop are unexamined.
3. **Intermittent unrelated failure observed once.** In one run of the full grep-selected set (run with `-x`, concurrently with the gap probe above, 184 s wall), `tests/unit/merge_service/test_search_deep_coverage.py::TestFetchTorrentDeep::test_fetch_torrent_client_error_reraises` failed. It did not reproduce in isolation (1 passed) nor in the subsequent full run (543 passed). It exercises `fetch_torrent/tenacity` retry, code this change does not touch; the failure text was not captured. Cause: UNKNOWN — candidate flake, should be tracked separately.
4. **\$11.4.214 linkage with BOB-145** remains the operator/tracker decision recorded on the row; this evidence does not resolve it.

Live soak addendum (conductor, 2026-09-23 ~18:57 CEST, stack recreated from HEAD 69b049a via `./start.sh --recreate`; containers healthy)

Command: `python3 soak.py — POST /api/v1/search` (query “ubuntu”) on the live merge service :7187, poll to completion while probing `GET /health` every 100 ms. Result (pasted): `search finished: completed results 1444 / probes=222 max=60ms p99=54ms >1s=0 errors=42` — every one of the 42 “errors” is `HTTPError 429 Too Many Requests` (the rate limiter answering the 10/s probe), returned within the same 60 ms bound; zero timeouts, zero probes over 1 s. Prior baseline recorded in the item: 2 of 141 probes dead in the 2026-08-21 soak. Now: 0 of 222 slow. ## Honest boundaries Single search, single soak, no live RED/ before run on the pre-fix container (the pre-fix behaviour is proven at unit level: 3 failed/3 passed on pre-fix source). Manual QA (\$11.4.185) still owed. Other loop-thread work (e.g. `_serialize_merged_rows` in `GET /search/{id}`) not audited; whether BOB-145 is the same defect (\$11.4.214) is left to the operator.